

Amdahl Reality Gap - Technical Analysis

Student ID	230107064
CPU	AMD Ryzen 7 5800HS with Radeon Graphics
Physical cores	8
Logical threads	16
Workload N	17,064,000
Sequential baseline T_seq	6.059 s
Derived parallel fraction p	0.9599 (95.99%)

Q1: Micro-Architectural Root Cause of False Sharing

The CPU uses a 64-byte cache line. In the naive variant, adjacent per-thread counters in `hit_count[]` can occupy the same cache line. When different cores repeatedly update their own counters on that shared line, the MESI/MOESI coherence mechanism must invalidate and transfer ownership of the line between cores, creating cache-coherence and invalidation traffic. On this machine, however, the measured penalty was very small: the naive version averaged 0.8605 s and the OpenMP reduction version averaged 0.8595 s. The penalty ratio was about 1.0012x, so this particular run did not show a large false-sharing slowdown.

Q2: Hyperthreading (SMT) Saturation and Physical Core Ceilings

The speedup did not continue linearly when the thread count increased from 8 physical cores to 16 logical threads. At 8 threads, the empirical speedup was about 6.45x. At 16 threads, it increased to about 9.81x, which is an improvement but far below a proportional doubling. SMT threads on the same physical core share hardware resources such as execution units, caches, instruction pipelines, and memory bandwidth, so two logical threads do not behave like two independent physical cores.

Q3: Empirical vs. Theoretical Amdahl Discrepancy

Using the measured 2-thread result, the derived parallel fraction was $p = 0.9599$, or about 95.99%. The theoretical Amdahl curve and the empirical curve diverged as the thread count increased because Amdahl's Law assumes idealized parallel execution and does not model several physical effects. Two important omitted factors are cache/memory contention and thread synchronization/scheduling overhead. At 16 threads, SMT resource sharing adds another practical limit. Small negative reality-gap values can also occur because empirical timings vary and the separate sequential baseline can differ slightly from the 1-thread parallel-code timing.

Q4: Scheduling Trade-Off Dilemma

On this machine, `schedule(dynamic,100)` averaged 0.9030 s, while `schedule(dynamic,10000)` averaged 0.8500 s and `schedule(static)` averaged 0.8815 s. Therefore, the small dynamic chunk size of 100 introduced enough queue-management and scheduling overhead to outweigh its load-balancing benefit. The larger dynamic chunk size of 10,000 performed better. `schedule(guided)` was the fastest measured option at 0.8275 s because it began with larger chunks and reduced them adaptively.

Key measured values

k	Avg T_k (s)	S_emp	S_theo	Delta
1	5.8010	1.0445	1.0000	-0.0445
2	3.1510	1.9229	1.9229	0.0000
4	1.7555	3.4514	3.5704	0.1190
8	0.9390	6.4526	6.2464	-0.2062
16	0.6175	9.8121	9.9901	0.1780